



3/6/14/9/10

2/3/6/10

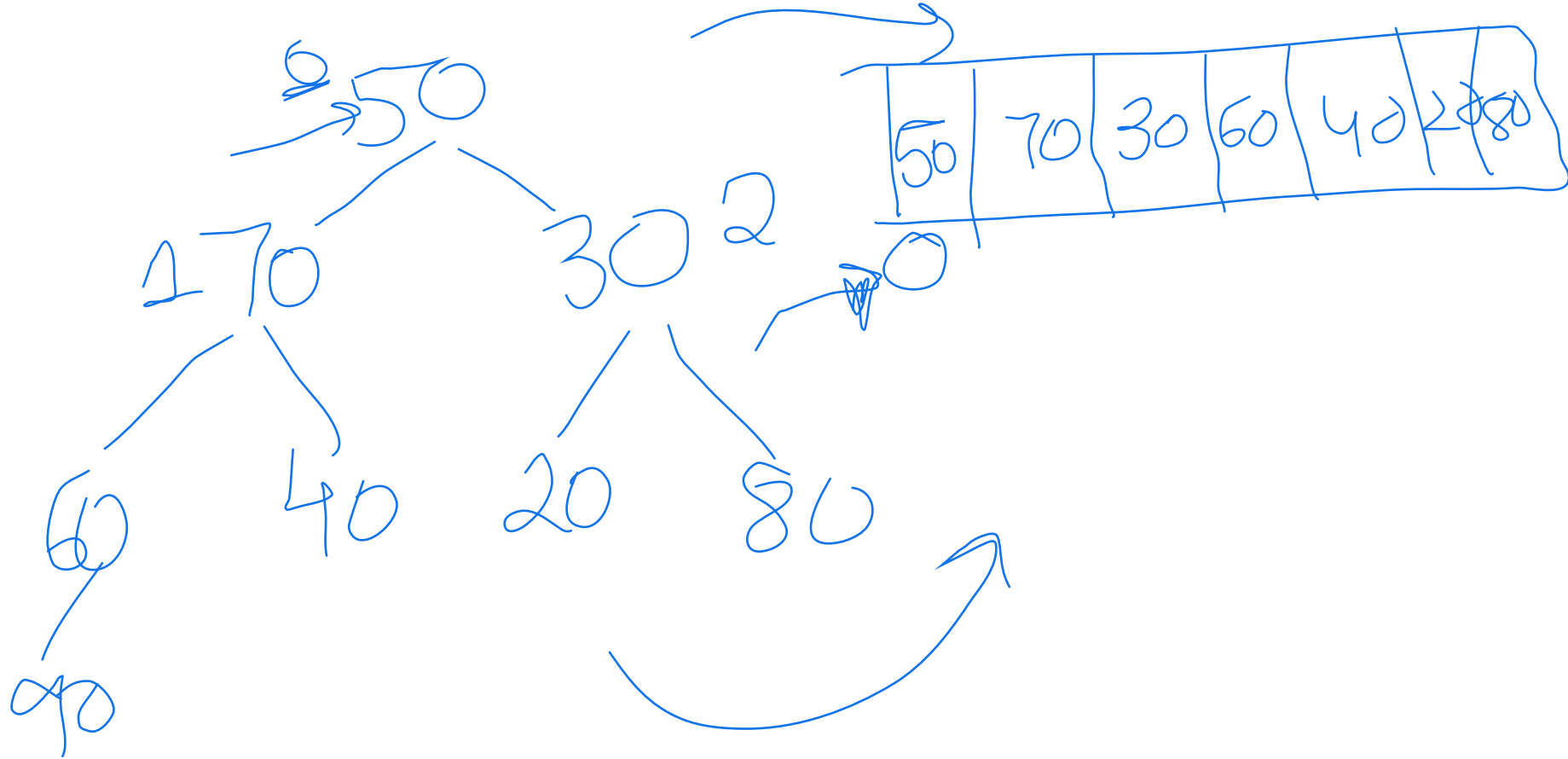
3 6 14 9 10 2 3 7 2

3/6/14/9/10/2/7/

Agenda

1. Binary Search Tree (Intro)
2. Insertion (Iterative)
3. Search (Recursive)
4. Deletion (Idea)
5. Analysis

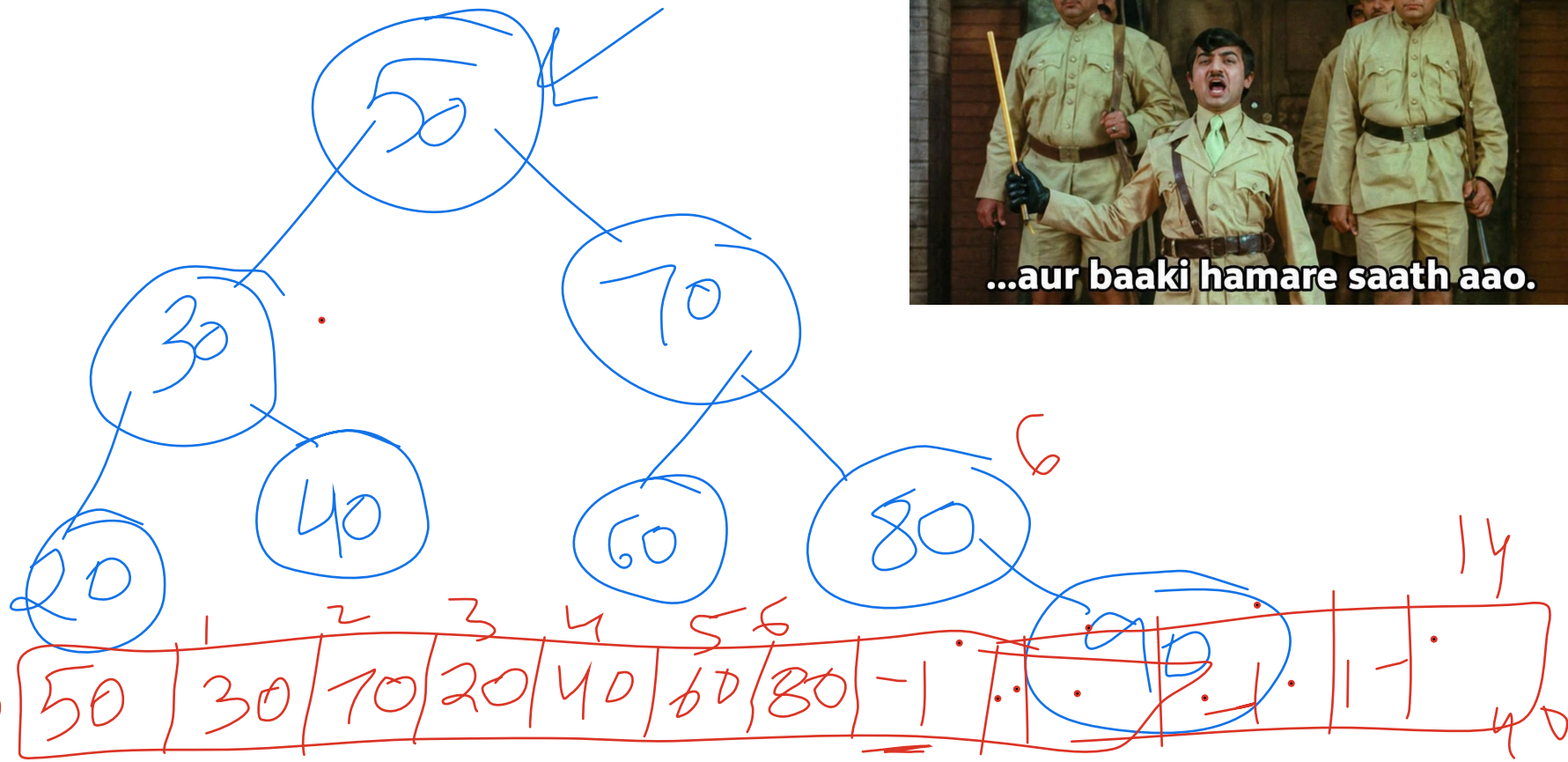
50, 70, 30, 60, 40, 20, 80, 90



50, 70, 30, 60, 40, 20, 80, 90



50, 70, 30, 60, 40, 20, 80, 90



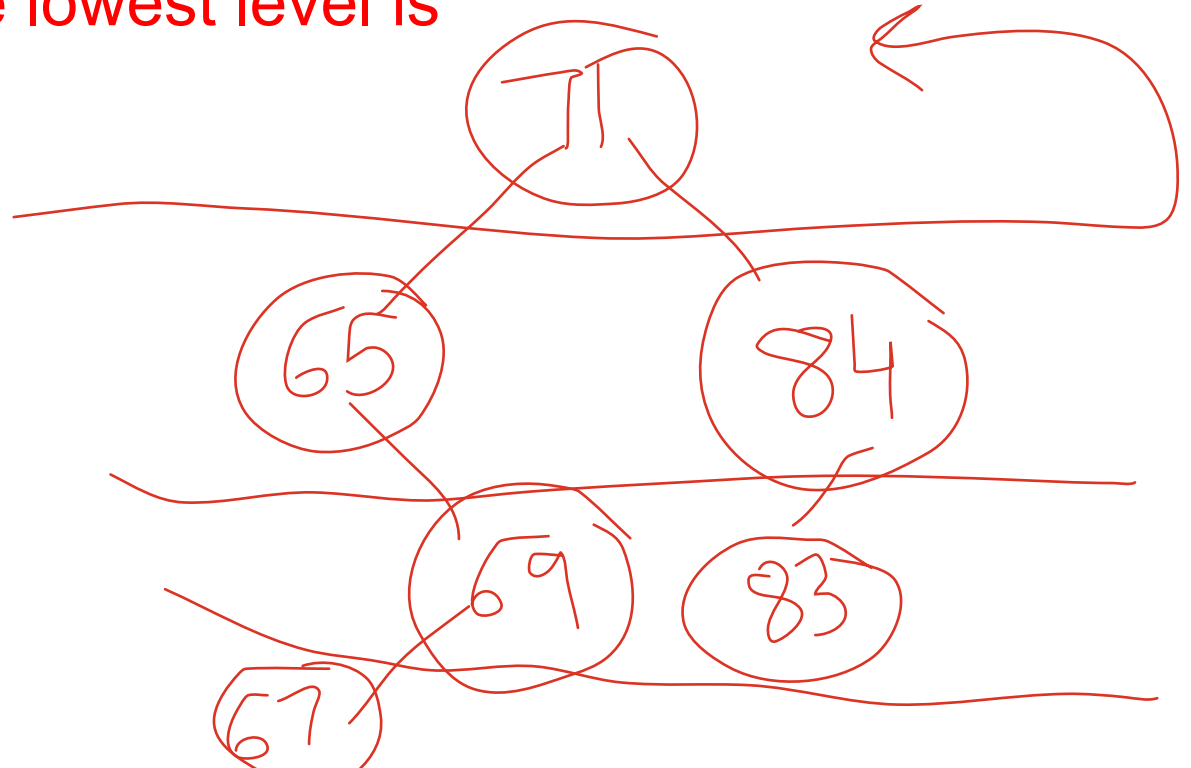
While inserting the elements 71, 65, 84, 69, 67, 83 in an empty binary search tree (BST) in the sequence shown, the element in the lowest level is

A, 65

B, 67

C, 69

D, 83



Insertion

Start at root.

Till an empty position is not found:

- If new value is smaller, go left.
- Otherwise, go right.

Insert the value at the empty position.

void insert(treenode* &root, int key)

treenode* ptr = root;

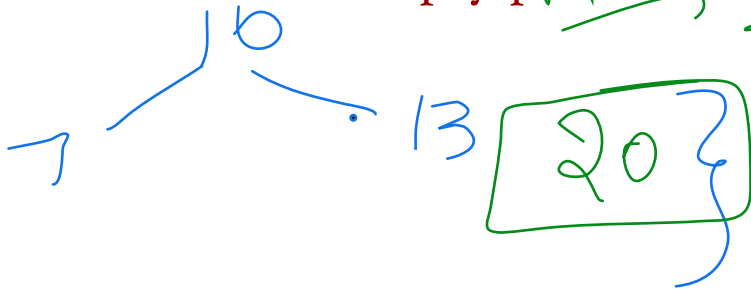
while(ptr != null)

{ if key < ptr->data

ptr = ptr->left

else ptr = ptr->right

}
create new node
newnode->data = key
if key < ptr->data ptr->left = newnode
else ptr->right = newnode



Insertion

Start at root.

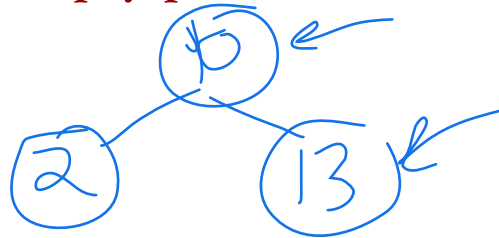


`void insert(int T[], int key)`

Till an empty position is not found:

- If new value is smaller, go left.
- Otherwise, go right.

Insert the value at the empty position.



`int i = 0;`
`while (T[i] != -1)`

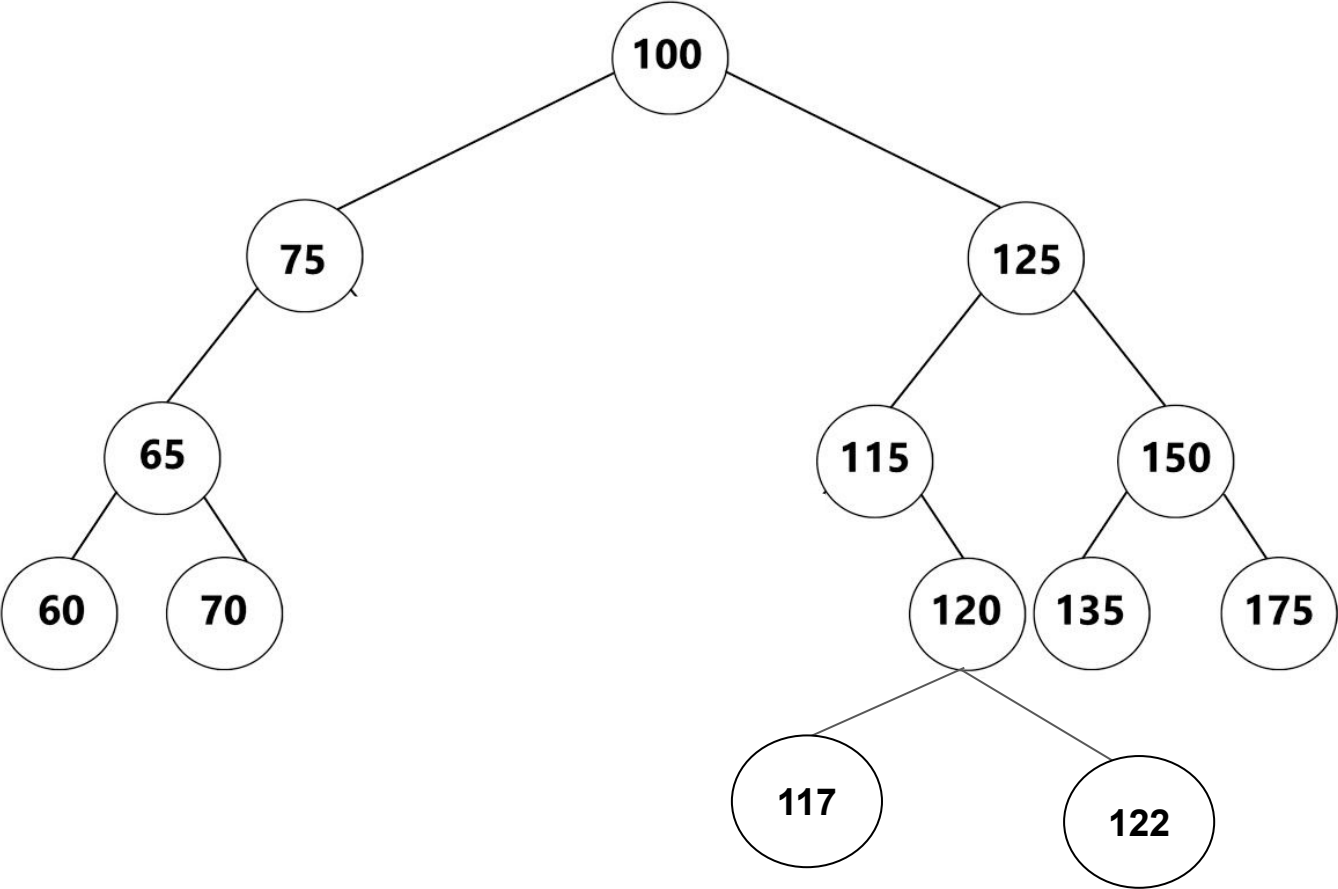
`{ if (key < T[i])`
`i = 2 * i + 1;`
`else i = 2 * i + 2;`
`T[i] = key;`

Search

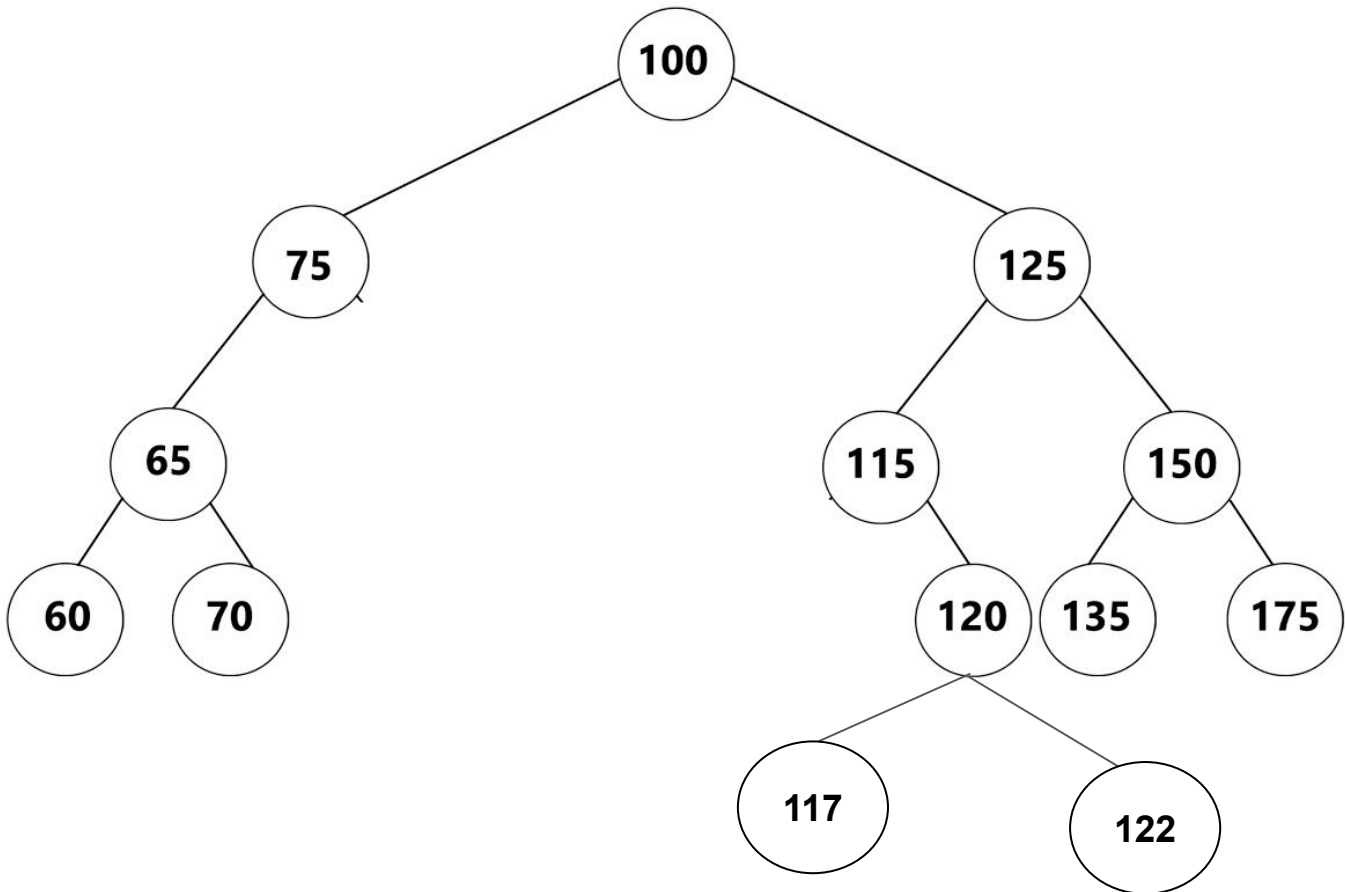
Function Search(tree, value, i)

```
{  
    If current position is empty: Value is not found  
  
    If current element is equal to the value: Value is found  
  
    If value is smaller than current element: Search in the left subtree  
  
    Otherwise: Search in the right subtree  
}
```

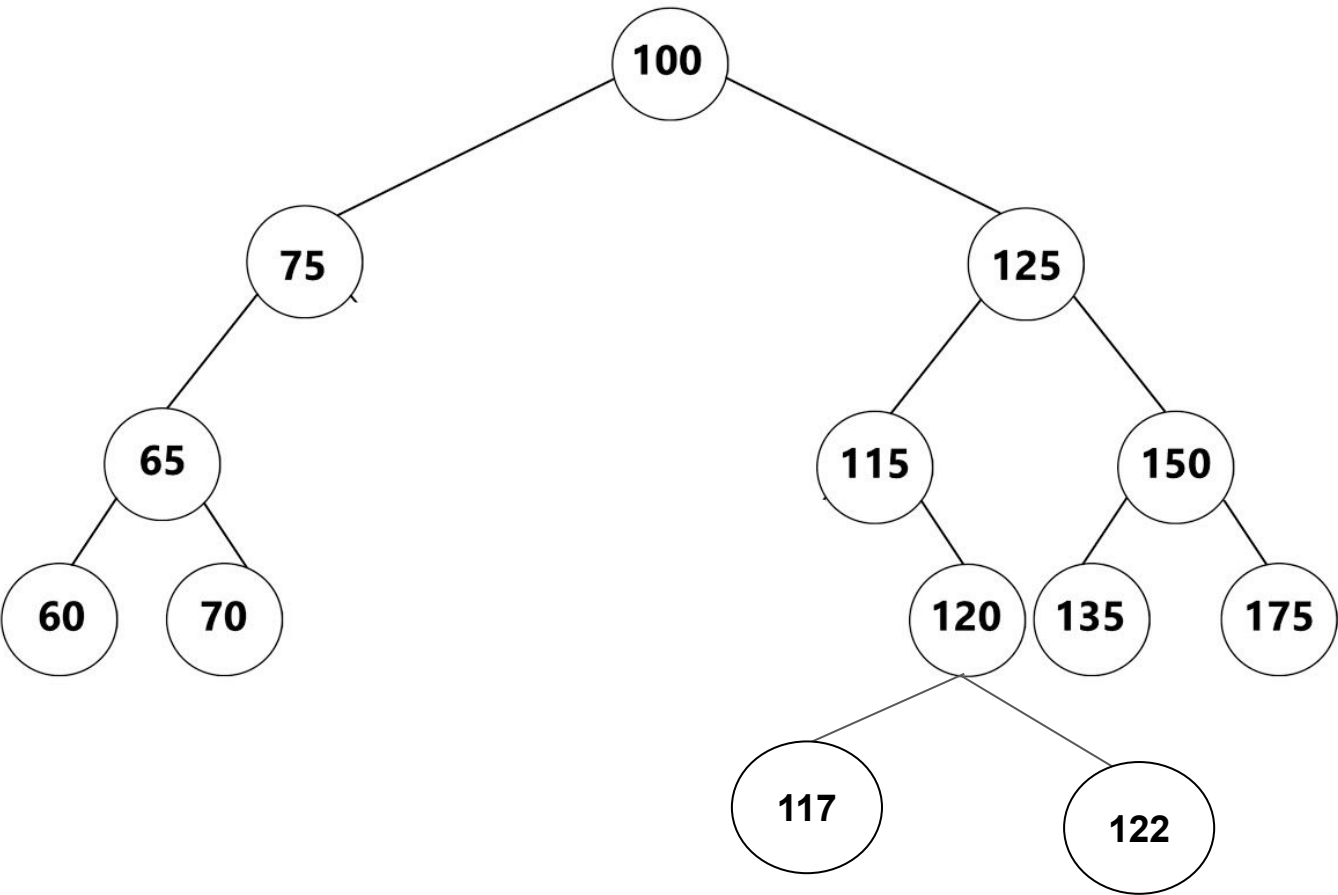
Deletion[CASE 1: Leaves]



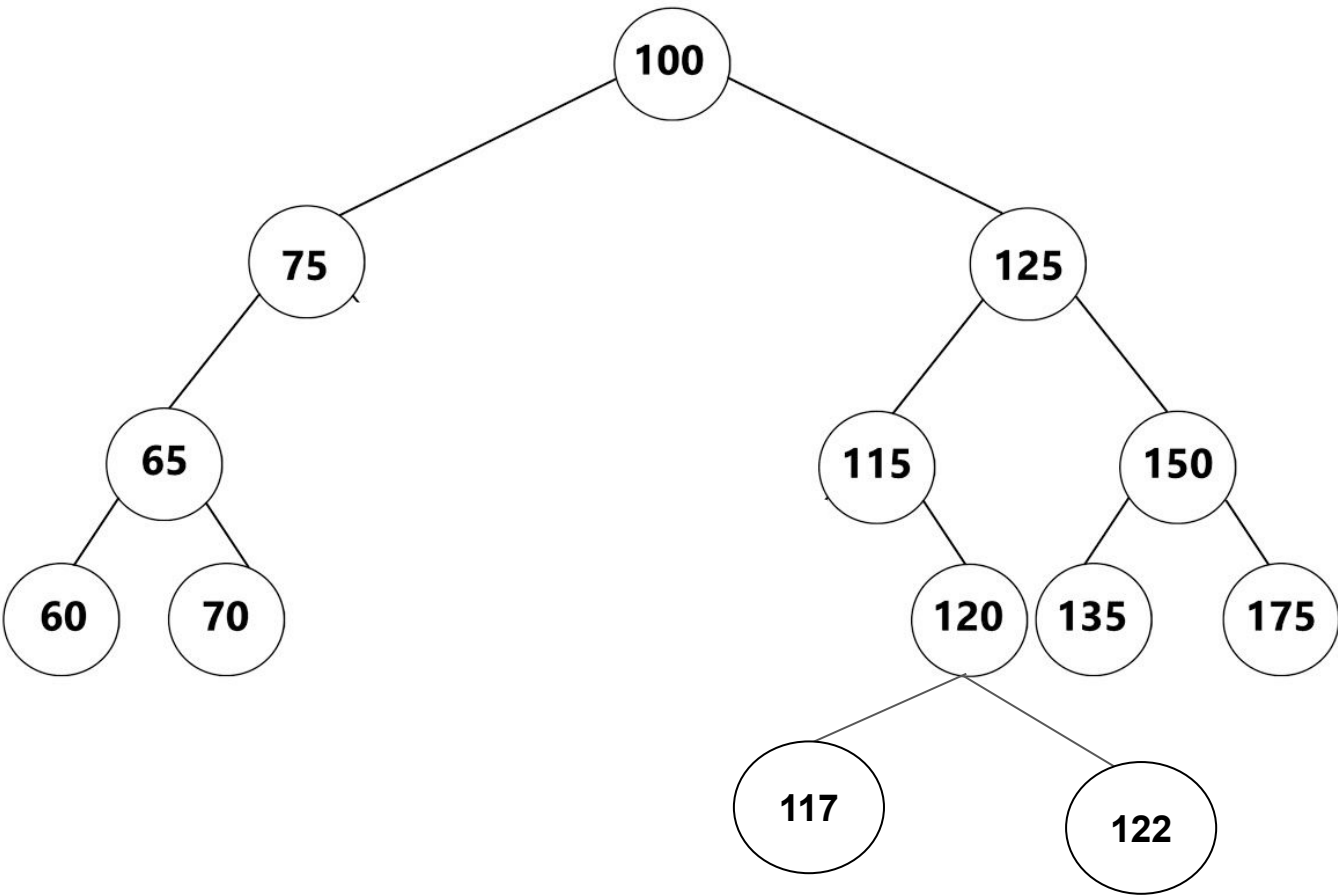
Deletion[CASE 2: Node with one child: 75]



Deletion[CASE 3: Node with 2 children: 100]

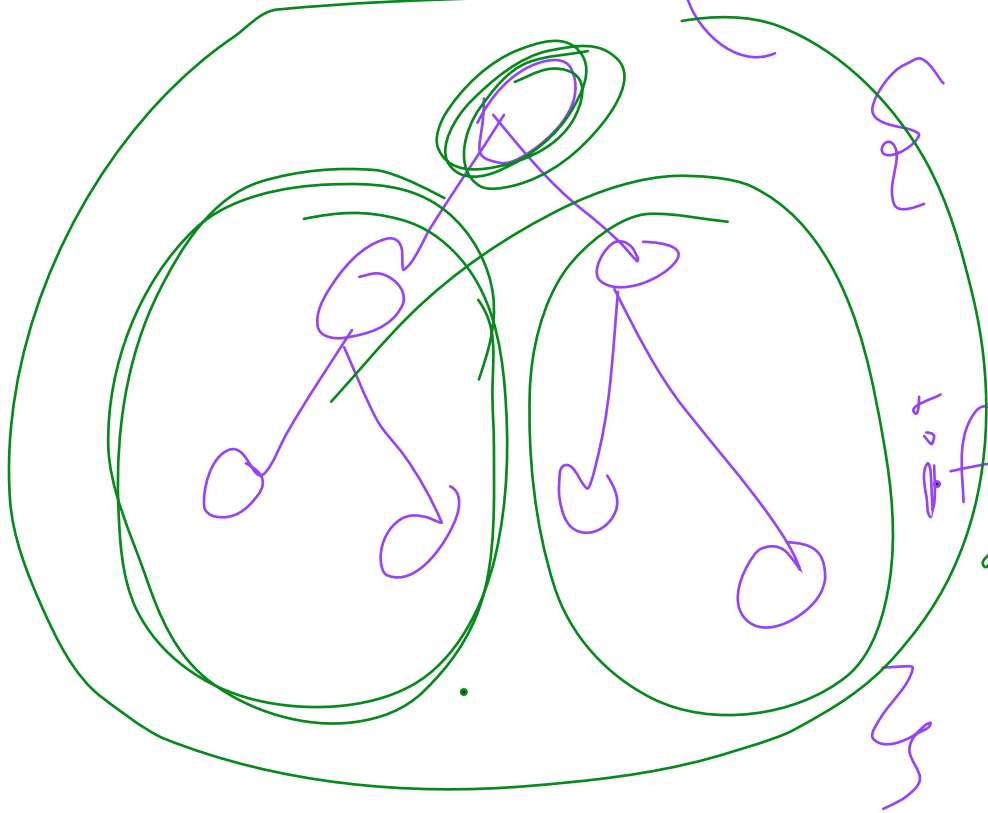


Deletion[CASE 3: Node with 2 children: 100]



~~Analysis~~

$\text{insert}(\text{tree}, \text{key}, \text{root})$

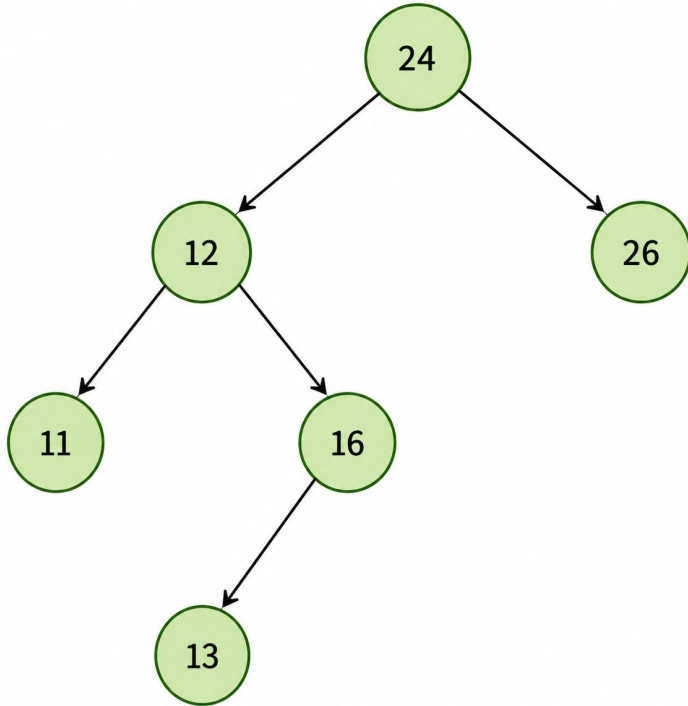


if there is no root
insert right here

if $\text{key} < \text{root's data}$

$\text{insert}(\text{tree}, \text{key}, \text{root} \rightarrow \text{left})$

If we remove the root node, which of the node from the left subtree will be the new root?



Draw the binary tree whose in-order traversal is DBEAFC and whose pre-order traversal is ABDECF.

A binary search tree when traversed in pre-order, the sequence of values obtained is 5, 3, 1, 2, 4, 6, 8, 7. If the tree is traversed in post-order, the sequence obtained would be